

La récursivité

I. Introduction

Dans certains cas, la solution d'un problème peut se ramener à résoudre le même problème de taille plus réduite, qui lui-même pourra se ramener à résoudre le même problème de taille encore plus petite ... jusqu'à aboutir à la résolution d'un problème élémentaire.

Exemple :

Soit la formule mathématique de la factorielle de n

- $n! = n * (n-1) * (n-2) * (n-3) * \dots * 1$
- et $0! = 1$

ou encore

- factorielle (n) = $n * (n-1) * (n-2) * (n-3) * \dots * 1$ **Formule 1**
- et factorielle (0) = 1

Si on utilise une fonction appelée factorielle on obtient :

- factorielle (n) = $n * \text{factorielle}(n-1)$ **Formule 2**
- factorielle (0) = 1

Constatations :

Pour définir factorielle (n) au niveau de la Formule 1, nous avons utilisé un traitement itératif.

Pour définir factorielle (n) au niveau de la Formule 2, nous avons utilisé un autre procédé répétitif : nous avons appelé la même fonction sur d'autres données plus simples (factorielle ($n-1$)). Ce procédé s'appelle Traitement Récursif.

II. Définitions et principes

II.1 Définitions

La récursivité est une méthode algorithmique qui consiste à appeler un sous-programme dans son propre corps.

Un sous-programme récursif est un module qui s'"appelle" lui-même. A chaque appel, il y a mémorisation des valeurs d'un ou plusieurs paramètres formels.

La limite technique de la récursivité est la mémoire stockant les données intermédiaires des sous-programmes, que nous appelons la "pile". Cette dernière déborde lorsque le nombre des appels devient important.

Remarques :

- 1) Les modules récursifs sont appelés depuis leur propre corps, soit directement, soit indirectement à travers un ou plusieurs autres modules.
- 2) La récursivité est dite croisée lorsque plusieurs procédures ou fonctions s'appellent mutuellement.

Un programme récursif doit :

- traiter un ou plusieurs cas particuliers représentés par une ou plusieurs conditions d'arrêt, sinon il tournera indéfiniment.
- traiter un ou plusieurs cas généraux représentés par des appels récursifs. Chaque fois qu'un programme récursif est appelé (par lui-même), un ou plusieurs des arguments qui lui sont transmis doivent converger et arriver à la condition d'arrêt.

II.2 Exemples

Pour calculer factorielle (4) ou 4!, nous procédons ainsi :

1ère étape : $4 \neq 0$ alors Factorielle (4) = $4 * \text{Factorielle (4-1)} = 4 * \text{Factorielle (3)}$

2ème étape : $3 \neq 0$ alors Factorielle (3) = $3 * \text{Factorielle (3-1)} = 3 * \text{Factorielle (2)}$

d'où Factorielle (4) = $4 * 3 * \text{Factorielle (2)}$

3ème étape : $2 \neq 0$ alors Factorielle (2) = $2 * \text{Factorielle (2-1)} = 2 * \text{Factorielle (1)}$

d'où Factorielle (4) = $4 * 3 * 2 * \text{Factorielle (1)}$

4ème étape : $1 \neq 0$ alors Factorielle (1) = $1 * \text{Factorielle (1-1)} = 1 * \text{Factorielle (0)}$

d'où Factorielle (4) = $4 * 3 * 2 * 1 * \text{Factorielle (0)}$

5ème étape : La valeur 0 est rejointe, c'est le cas particulier qui provoquera l'arrêt des étapes du calcul de la factorielle, alors Factorielle (0) = 1

D'où Factorielle (4) = $4 * 3 * 2 * 1 * 1$

Le résultat final = 24

Conclusion :

Voici la solution récursive de la fonction Factorielle :

Fonction Factorielle (n : Entier) : Entier

Var

F : entier

Début

Si (n = 0) Alors

F ← 1

Sinon

F ← n * Factorielle (n-1)

Fin Si

Retourne (F)

Fin Factorielle

II.3 Activités

Exercice 1

Proposez une analyse, puis déduisez l'algorithme **itératif** d'une fonction permettant de vérifier si une chaîne de caractères CH est un palindrome.

Exemples : radar, rotor, été.

Analyse de la fonction Palindrome

Une structure itérative à condition d'arrêt (Répéter ... Jusqu'à) vérifiant les actions suivantes :

- À chaque itération, nous comparons deux à deux les caractères symétriques de CH relatifs à une position i ($\text{Verif} \leftarrow (\text{ch}[i] = \text{ch}[\text{Long}(\text{ch}) - i + 1])$).
- Nous arrêtons la répétition à la première différence ($\text{Verif} = \text{Faux}$) ou lorsque nous atteignons les caractères du milieu ($i > \text{Long}(\text{ch}) \text{ Div } 2$).

Algorithme itératif de la fonction Palindrome

```

Fonction Palindrome (ch : Chaîne) : Booléen
Var
    Verif : booléen
Début
    i ← 1
    Répéter
        Verif ← (ch[i] = ch[Long(ch) - i + 1])
        i ← i + 1
    Jusqu'à (i > Long(ch) Div 2) Ou (Verif = Faux)
    retourne ( verif )

```

Fin Palindrome

Exercice 2

Dégagez une relation **récursive** permettant de vérifier si une chaîne de caractères CH est un palindrome ou non.

➤ prenons l'exemple de la chaîne $\text{ch} = \text{"radar"}$, de longueur = 5

1ère étape : $\text{Long}(\text{"radar"}) = 5$; comme 5 n'est pas inférieur à 2, et comme les caractères extrêmes de la chaîne "radar" sont identiques = "r" alors :

$\text{Palindrome}(\text{"radar"}) = \text{Palindrome}(\text{"ada"})$

2ème étape : $\text{Long}(\text{"ada"}) = 3$; comme 3 n'est pas inférieur à 2, et comme les caractères extrêmes de la chaîne "ada" sont identiques = "a" alors,

$\text{Palindrome}(\text{"ada"}) = \text{Palindrome}(\text{"d"})$

3ème étape : $\text{Long}(\text{"d"}) = 1$; comme 1 est inférieur à 2, c'est le cas particulier qui provoque l'arrêt des étapes de la vérification si ch est palindrome ou non, alors $\text{Palindrome} = \text{Vrai}$

Dans cet exemple, ch est un palindrome car on a obtenu une chaîne formée d'un seul caractère ("d") sans rencontrer de i vérifiant $\text{ch}[i] \neq \text{ch}[\text{Long}(\text{ch}) - i + 1]$.

Remarques : Un mot d'un seul caractère est un palindrome, sinon il l'est dès que ses caractères extrêmes sont les mêmes et le mot restant est aussi un palindrome.

➤ d'où, la relation récursive suivante :

Si Longueur (ch) < 2 alors

ch est un palindrome

Sinon

si $ch[1] = ch[\text{Long}(ch)]$ alors

pal $\leftarrow$ Palindrome (ch sans ses premier et dernier caractères)

Sinon ch n'est pas un palindrome.

Algorithme :

Fonction Palindrome (ch : Chaîne; d , f : entier) : Booléen

Var

test : booléen

Début

Si (f- d) < 1 Alors

test $\leftarrow$ Vrai

Sinon

Si $ch[d] = ch[f]$ Alors

test $\leftarrow$ Palindrome (ch, d+1, f-1)

Sinon

test $\leftarrow$ Faux

FinSi

Retourne (test)

Fin Palindrome

Conclusion :

La solution récursive comporte :

Un appel récursif en changeant la valeur du paramètre. Dans l'exemple de la fonction Palindrome, le paramètre ch est démunie de son premier et de son dernier caractère s'ils sont identiques,

(test $\leftarrow$ Palindrome (ch, d+1, f-1))

Une 1 ère condition d'arrêt de l'appel récursif. En effet, nous arrêtons le traitement lorsque les caractères extrêmes du paramètre sont différents (test $\leftarrow$ Faux)

Une 2ème condition d'arrêt de l'appel récursif, lorsque le paramètre est composé d'un seul caractère (Si (f – d) < 1 Alors test $\leftarrow$ Vrai)

